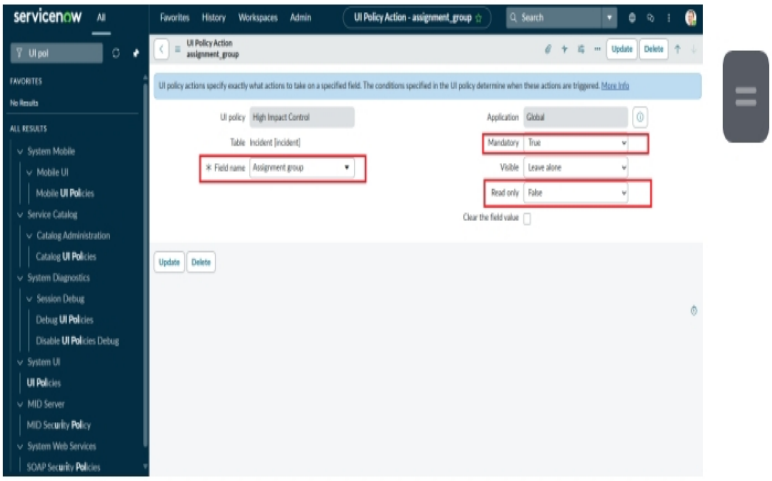
PROJECT DEMONSTRATION

ServiceNow – Implement Client Script & UI Policy (Incident)



Purpose: Ready-to-submit project demonstration document prepared from the provided demonstration video.

Platform: ServiceNow Developer Instance

Module: Incident / UI Policies / Client Scripts

This PDF contains the project flow, configuration values, scripts, verification steps, and screenshots that can be used as the project demonstration submission.

1. Project Demonstration – What to Include

The demonstration implements dynamic behavior and validation for Incident records in ServiceNow. The work is divided into five tasks.

Task	Demonstration	Main configuration / result
1	Create UI Policy on Incident	High Impact Control; condition: Impact is 1 – High
2	Create UI Policy Action – Urgency	Urgency becomes Read-only when policy condition is true
3	Create onChange Client Script	When Impact becomes High, Urgency is set to High automatically
4	Create onSubmit Client Script	High-impact incident cannot be saved without Assigned To
5	Create onCellEdit Client Script	Prevents direct State changes through list editing

Submission screenshot checklist

- Screenshot of the UI Policy configuration for High Impact Control.
- Screenshot of the UI Policy Action for Urgency (Read-only = True).
- Screenshot showing Incident Impact = High and the resulting Urgency / Priority behavior.
- Screenshot of the onChange Client Script and its script editor.
- Screenshot of the onSubmit Client Script and its script editor.
- Screenshot of the onCellEdit Client Script and its script editor.
- Optional verification screenshots showing the expected behavior on an Incident record or list.

2. Task 1 – Create UI Policy on Incident

UI Policy name: High Impact Control

Table: Incident

Active: True

Condition: Impact is 1 – High

Reverse if false: True

Purpose: Apply additional controls to an Incident when its Impact is High.

Steps

• Go to System UI → UI Policies.

• Click New.

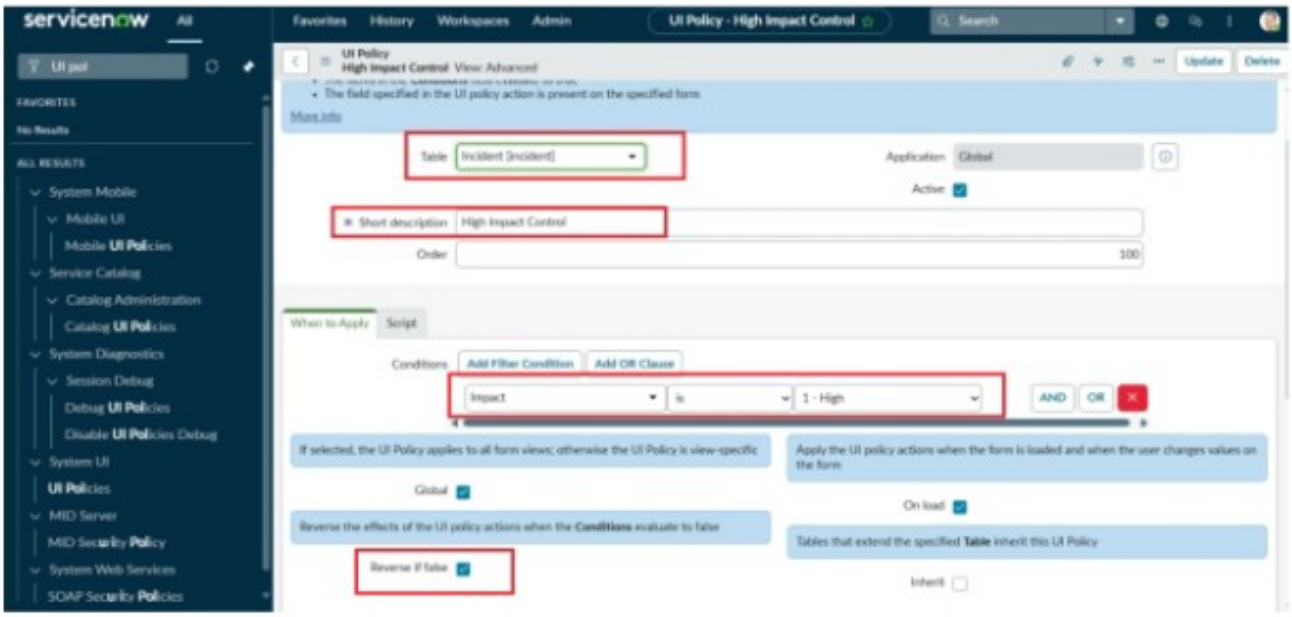
• Enter the name High Impact Control and select the Incident table.

• Set Active to True.

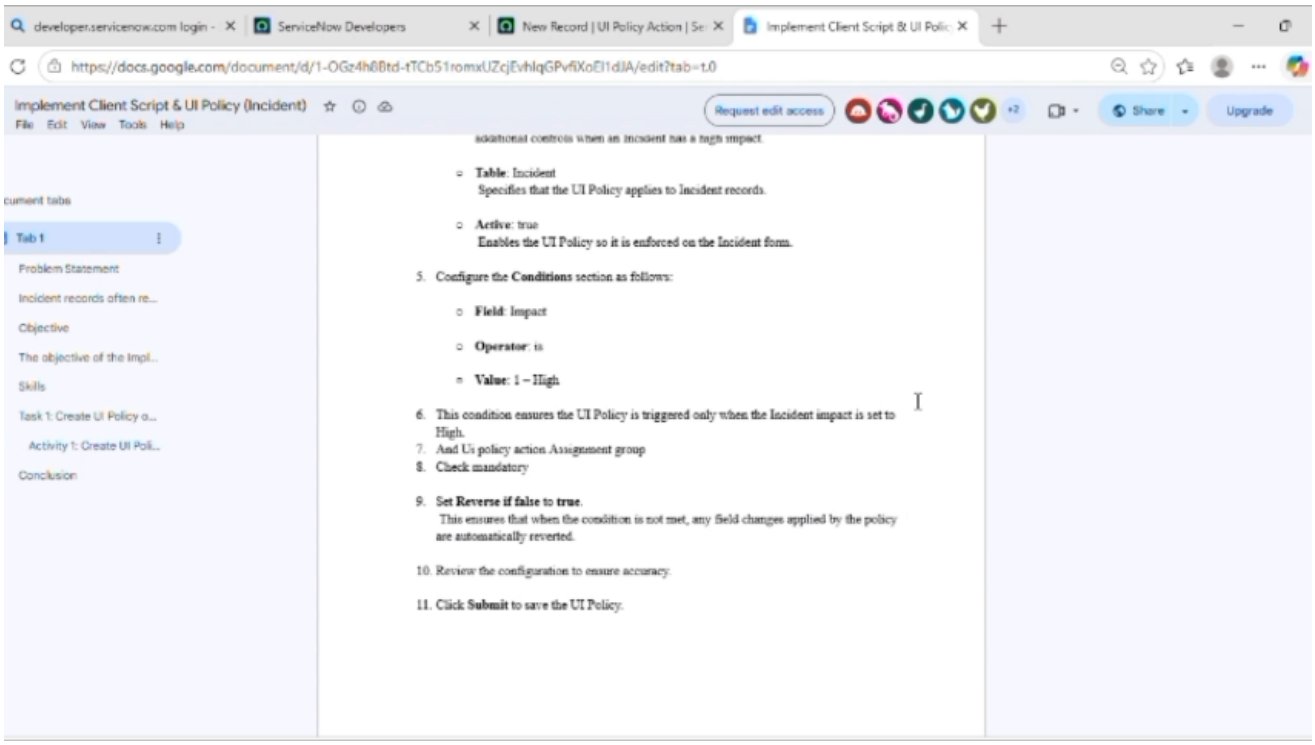
• Add the condition Impact is 1 – High.

• Enable Reverse if false so the UI Policy actions are reverted when the condition becomes false.

• Save / Submit the UI Policy.



Screenshot 1 – ServiceNow UI Policy form showing Incident table, High Impact Control, Active status and Conditions section.



Screenshot 2 – Demonstration document showing the required condition and Reverse if false setting.

3. Task 2 – Create UI Policy Action for Urgency

UI Policy: High Impact Control

Table: Incident

Field name: Urgency

Read-only: True

Visible: Leave alone

Mandatory: Leave alone

Steps

• Open the High Impact Control UI Policy and scroll to the UI Policy Actions related list.

• Click New to create a UI Policy Action.

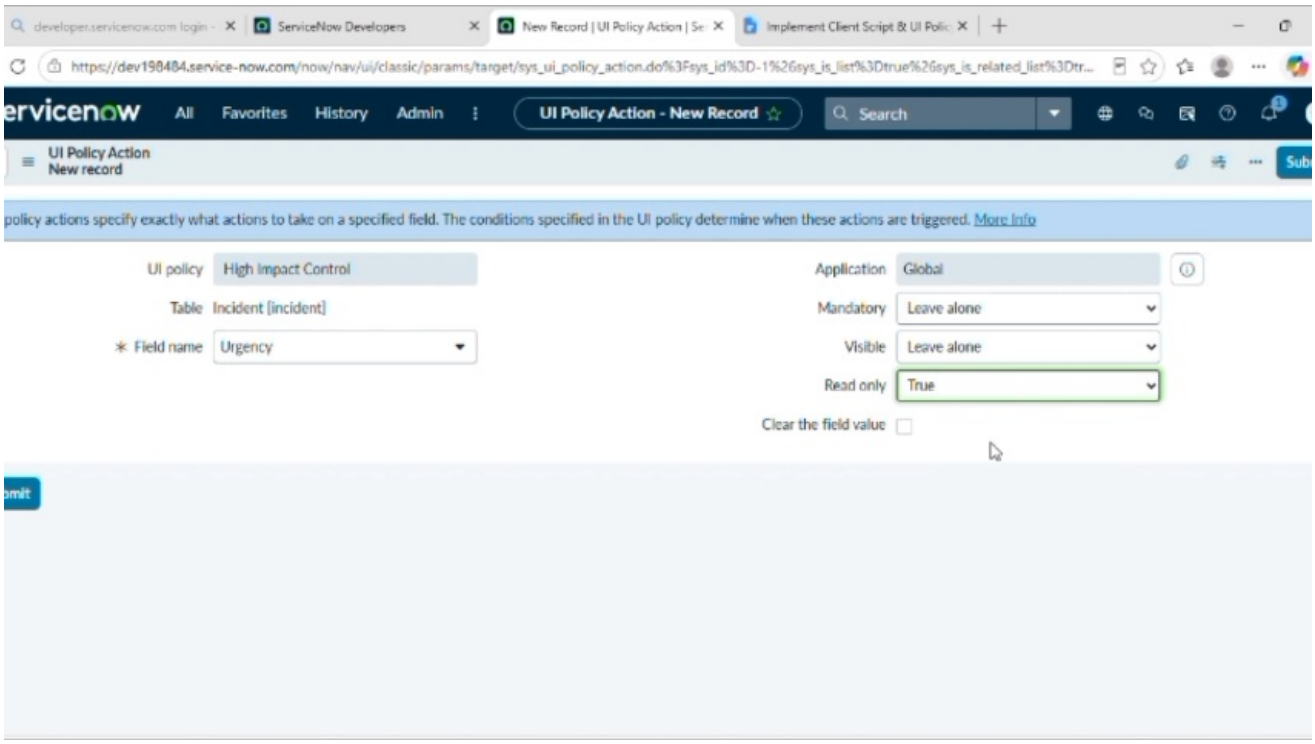
• Select Urgency as the field name.

• Set Read-only to True.

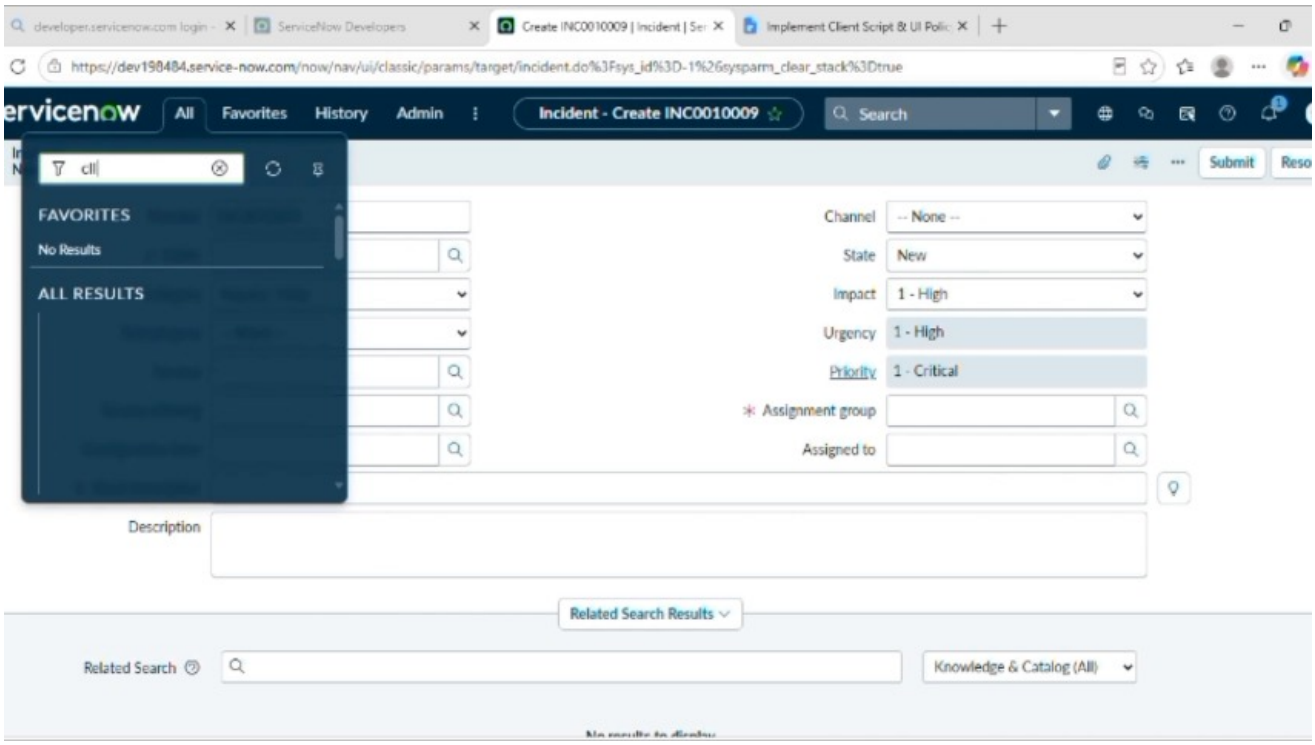
• Leave Mandatory and Visible unchanged unless the project instructions require otherwise.

• Submit the action.

• Verify by opening an Incident, setting Impact to High, and checking that Urgency becomes read-only.



Screenshot 3 – UI Policy Action for High Impact Control with Urgency selected and Read-only = True.



Screenshot 4 – Incident demonstration showing Impact = 1-High and the resulting Urgency / Priority values; Assignment group is also shown as mandatory.

4. Task 3 – Create onChange Client Script

Name: Auto set urgency for high impact

Table: Incident

Type: onChange

Field name: Impact

Active: True

Purpose: When Impact changes to High, automatically set Urgency to High and display an information message.

Script

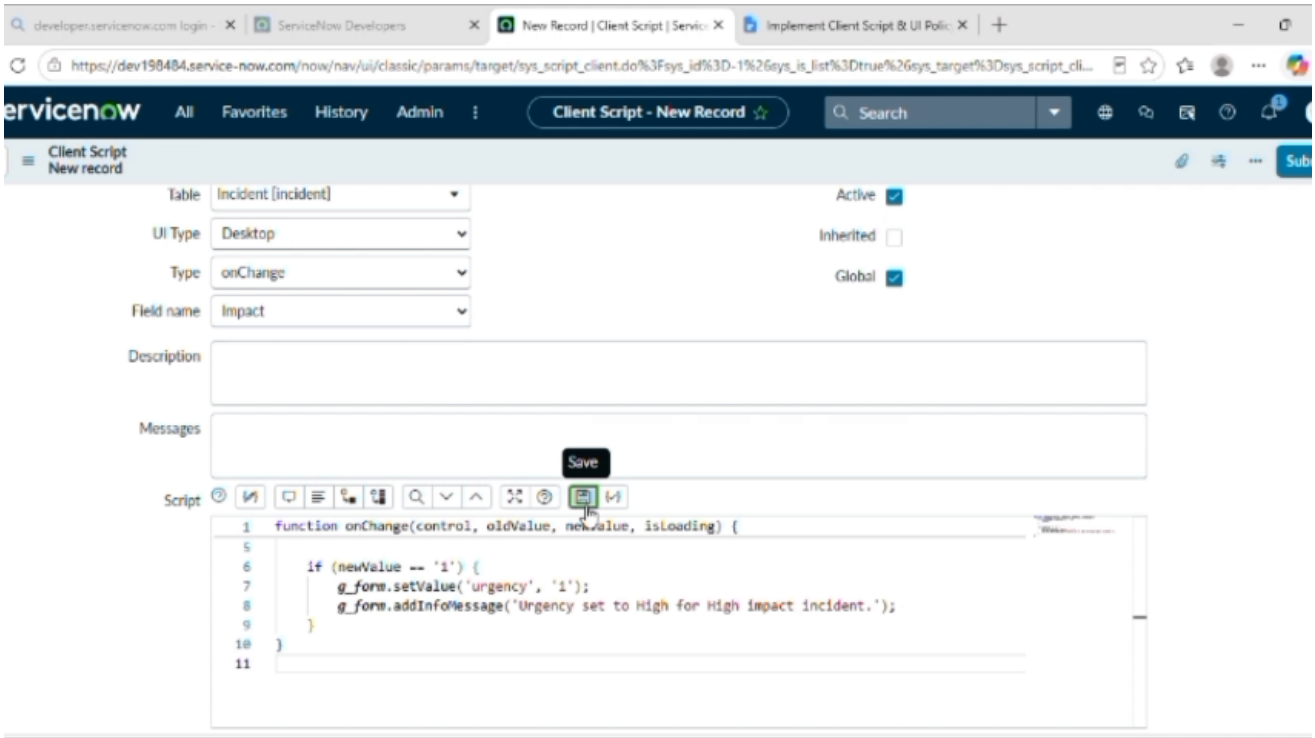
```
function onChange(control, oldValue, newValue, isLoading) {
  if (newValue == '1') {
    g_form.setValue('urgency', '1');
    g_form.addInfoMessage('Urgency set to High for High impact incident.');
  }
}
```

Verification

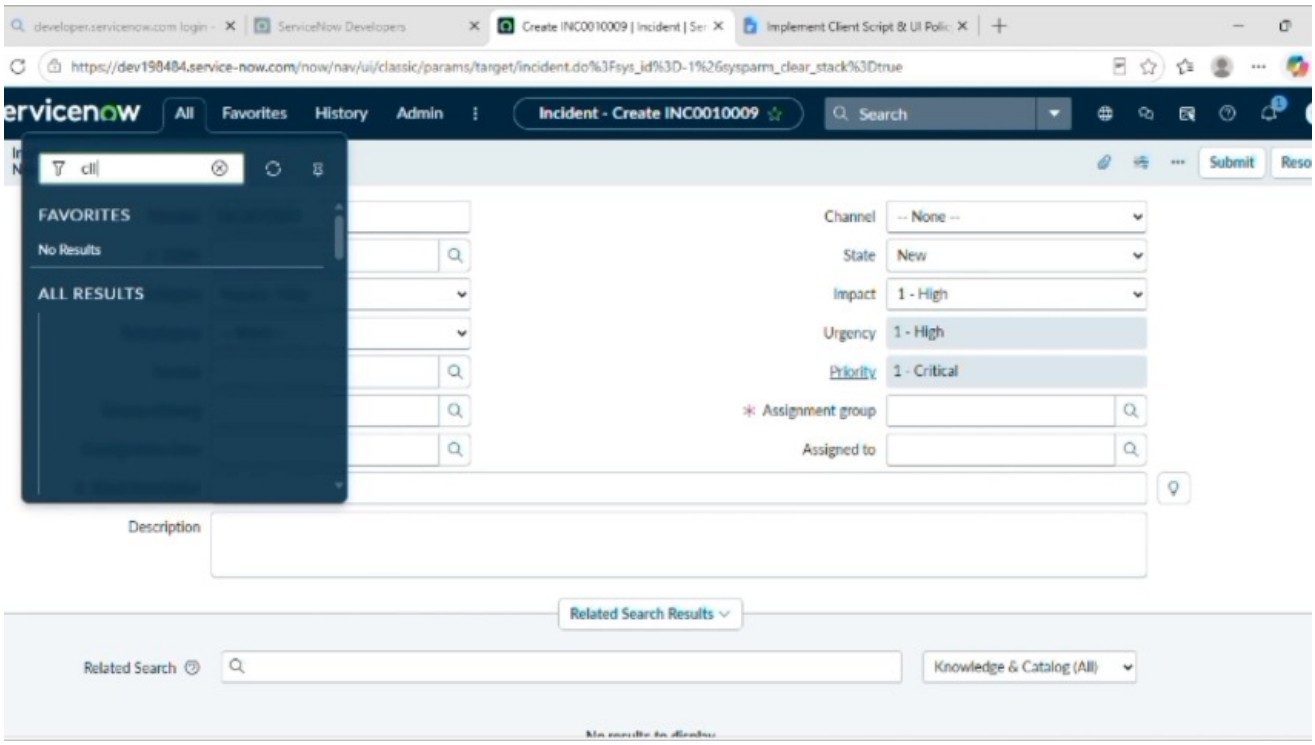
• Open an Incident record.

• Change Impact to High.

• Confirm that Urgency changes to High automatically.



Screenshot 5 – onChange Client Script form showing Incident table, onChange type, Impact field and script editor.



Screenshot 6 – Demonstration result after changing Impact to High.

5. Task 4 – Create onSubmit Client Script

Name: Prevent save if Assigned To missing

Table: Incident

Type: onSubmit

Active: True

Purpose: Prevent saving a High-impact Incident when the Assigned To field is empty.

Script

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {
    g_form.showErrorBox('assigned_to',
      'Assigned To is mandatory for High impact incidents.');
    return false;
  }
  return true;
}
```

Steps and verification

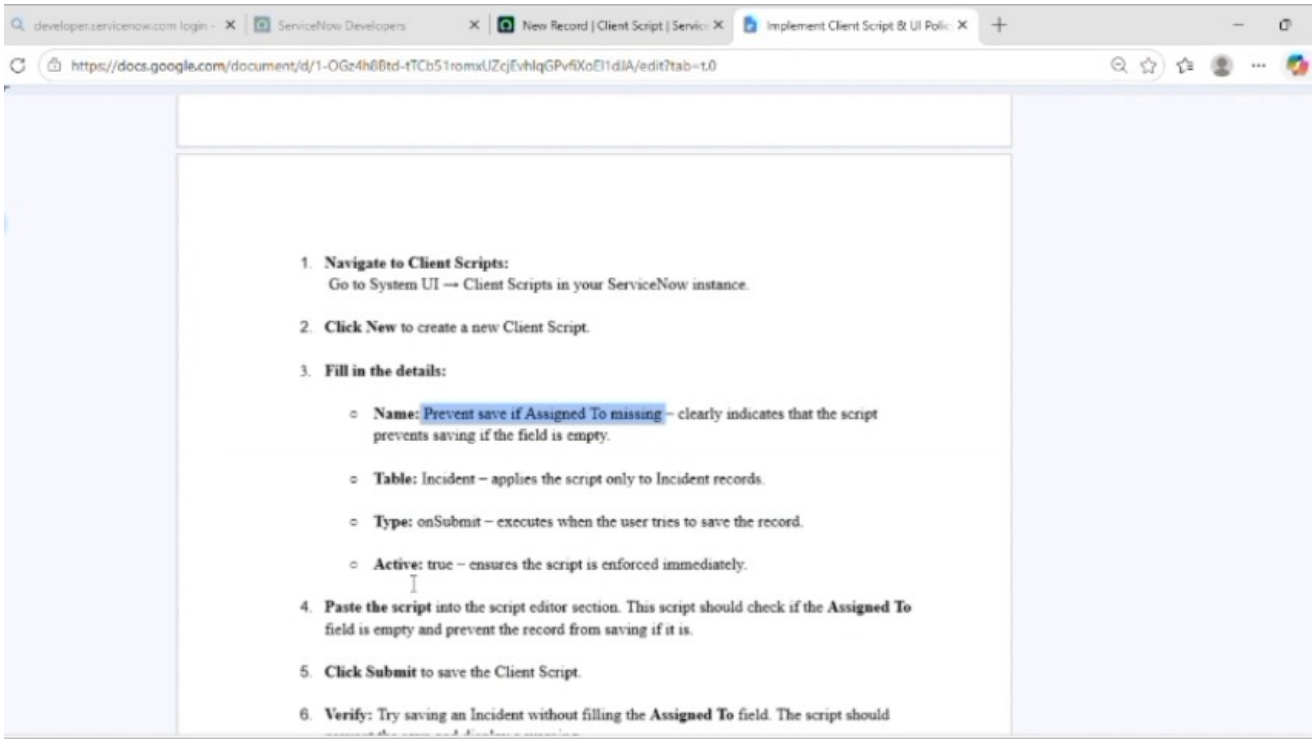
• Go to System UI → Client Scripts and click New.

• Enter the name, Incident table, onSubmit type and Active = True.

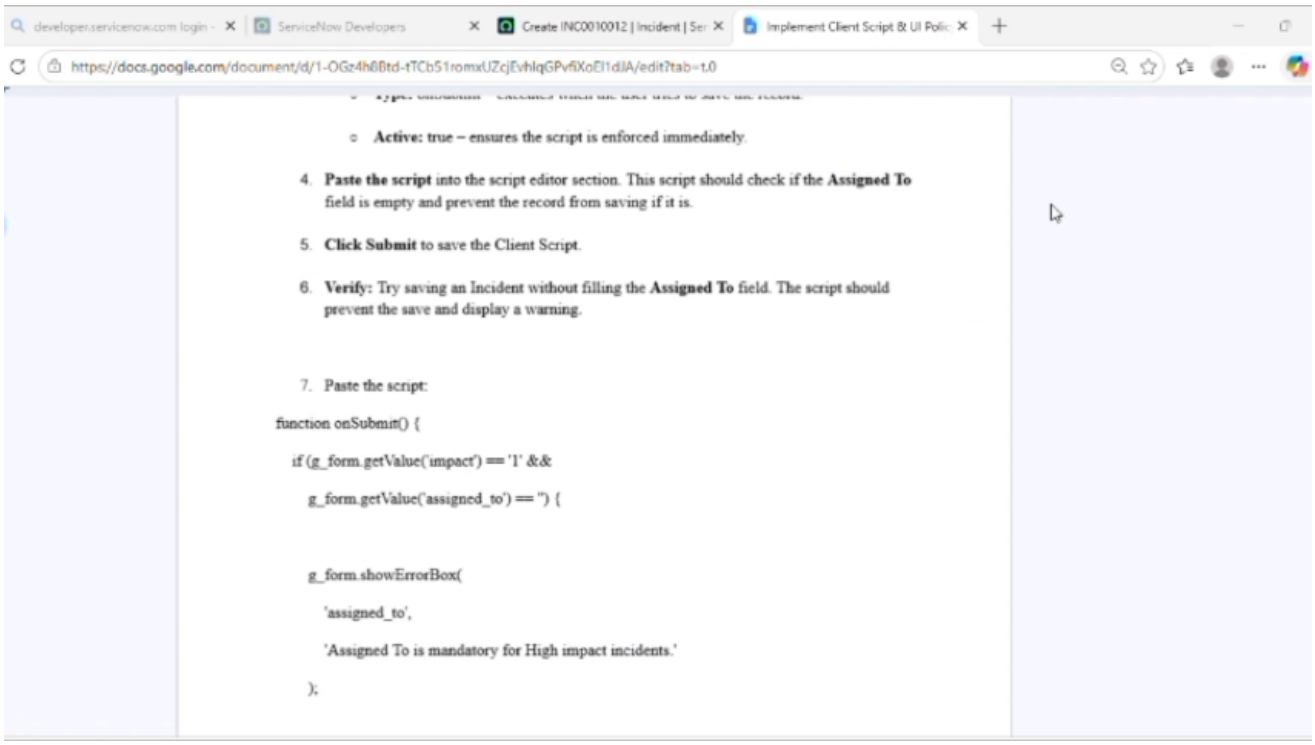
• Paste the script into the Script field and Submit.

• Try saving a High-impact Incident without filling Assigned To.

• The record should not save and an error message should be displayed.



Screenshot 7 – Demonstration instructions for the onSubmit Client Script.



Screenshot 8 – Script shown in the demonstration document for preventing save when Assigned To is missing.

6. Task 5 – Create onCellEdit Client Script

Name: Prevent state change via list edit

Table: Incident

Type: onCellEdit

Field name: State

Active: True

Purpose: Prevent unauthorized changes to the State field when users edit an Incident directly from a list view.

Script

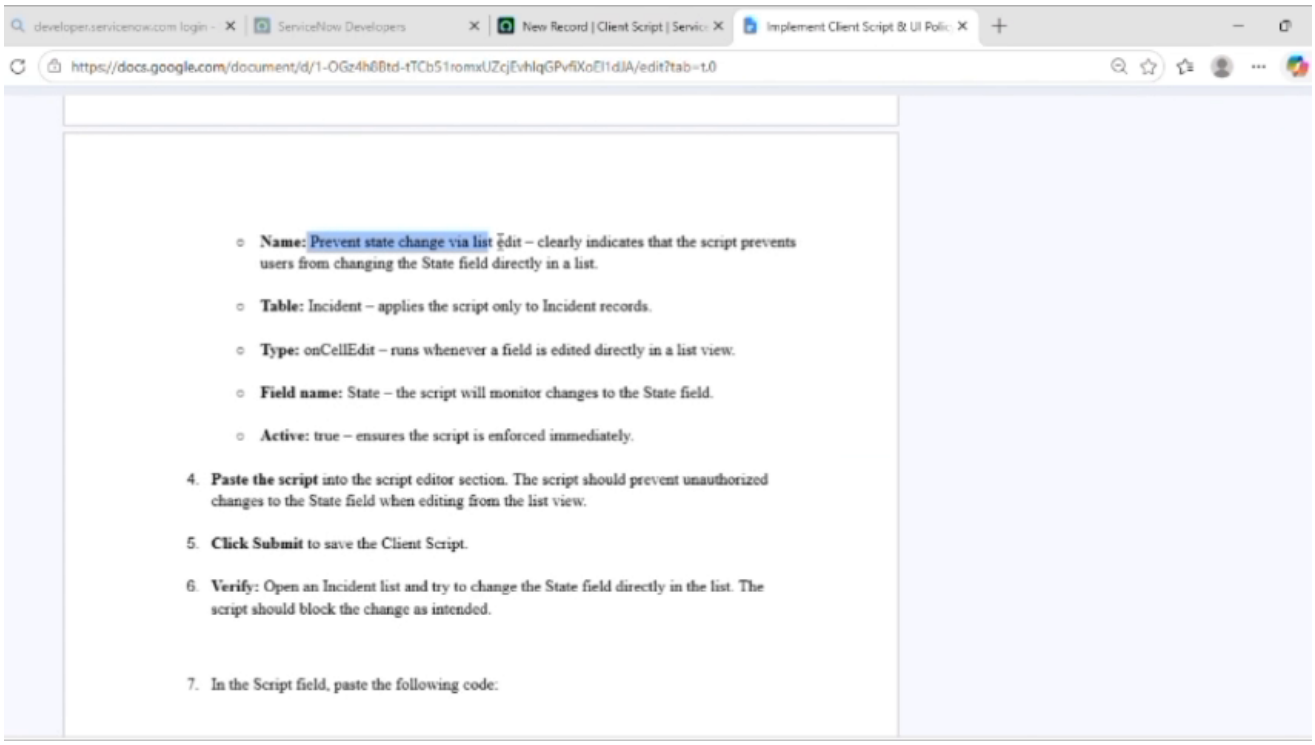
```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
  alert('State cannot be updated using list editing. Please open the Incident.');
  callback(false);
}
```

Verification

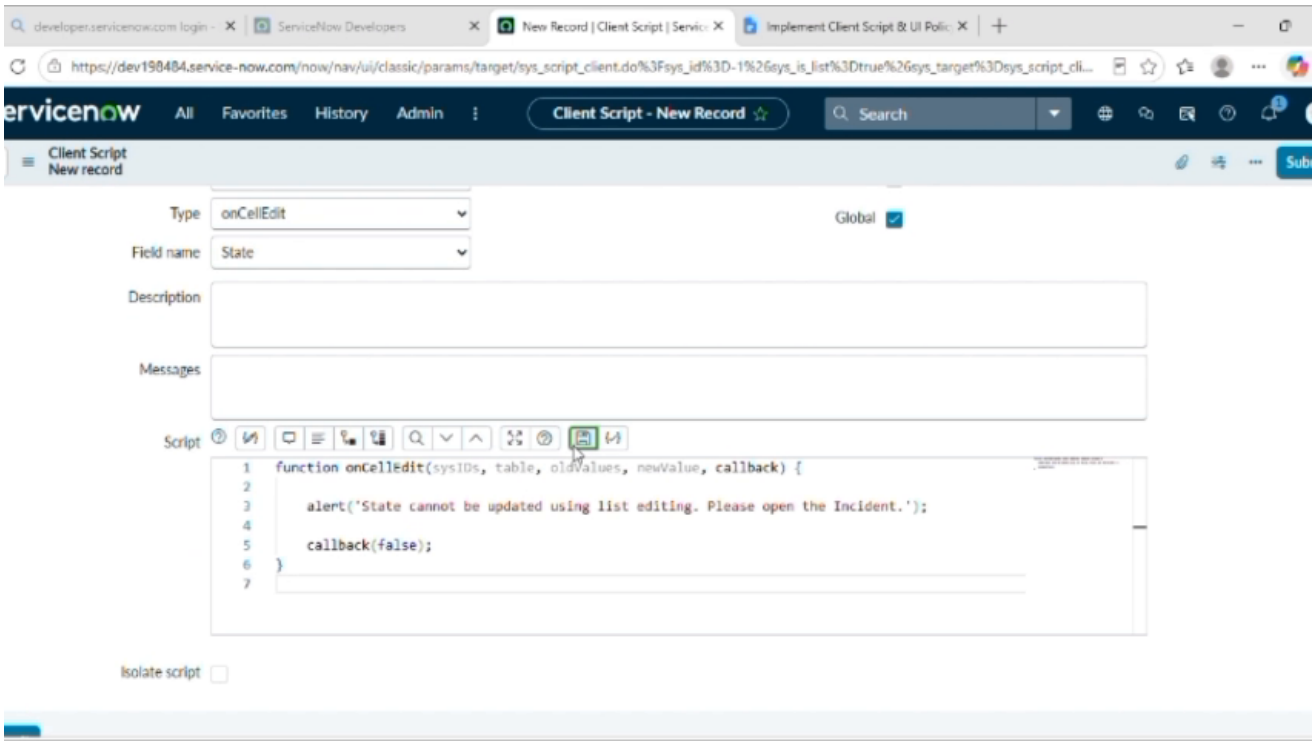
• Open an Incident list.

• Try to change the State field directly in the list.

• The script should block the change and display the warning message.



Screenshot 9 – Demonstration instructions for the Prevent state change via list edit Client Script.



Screenshot 10 – ServiceNow Client Script form showing onCellEdit, State field and the callback(false) script.

7. Final Project Submission Checklist

Before uploading this PDF to the project submission portal, check that the following items are included.

✓	Item to check
☐	Project title and platform are mentioned.
☐	Task 1 – High Impact Control UI Policy is documented.
☐	Impact is set to 1 – High and Reverse if false is enabled.
☐	Task 2 – Urgency UI Policy Action is documented.
☐	Urgency Read-only = True is shown.
☐	Task 3 – onChange Client Script is documented with screenshot and code.
☐	Task 4 – onSubmit Client Script is documented with screenshot and code.
☐	Task 5 – onCellEdit Client Script is documented with screenshot and code.
☐	Verification / expected result is written for each task.
☐	PDF is saved and uploaded as the project demonstration document.

8. Conclusion

The project demonstrates how ServiceNow UI Policies and Client Scripts can control Incident form behavior, automate field values, validate submissions, and restrict direct list editing. The screenshots in this PDF are extracted from the supplied demonstration video and arranged according to the five project tasks.

Ready for submission: this PDF is structured as a project demonstration record and can be uploaded along with the project, subject to your institution's submission requirements.